

Tutorial 3/10 – Python for Data Science

NumPy: The Art of Thinking in Arrays

STOP LOOPING. START DESCRIBING.

The mental shift that makes your Python actually fast — and makes Pandas make sense.

```
for s in scores:  
    result.append(  
        s * 2)
```

scores * 2

THE PERFORMANCE PROBLEM

Why Python Loops Aren't Enough

PYTHON LOOP

12.0s

1,000,000 numbers · flexible, interpreted, slow

NUMPY

0.08s

Same result · compiled, contiguous, vectorized

SAME RESULT · 50-200× FASTER — STRUCTURAL, NOT MARGINAL

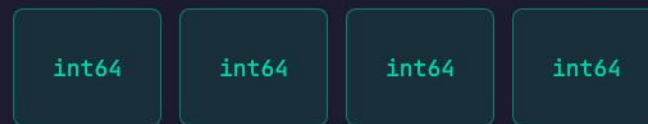
NDARRAY: THE DIFFERENT TRADEOFF

What NumPy Arrays Actually Are

Python list – pointer-chasing



NumPy ndarray – contiguous memory



Same type, packed tight → operations run in compiled **C and Fortran**.
Foundation of Pandas · scikit-learn · TensorFlow · PyTorch.

3 Ways to Create Arrays

```
import numpy as np

# From a Python list – most common starting point
scores = np.array([85, 92, 78, 95, 88])
print(scores)          # [85 92 78 95 88]
print(type(scores))    # <class 'numpy.ndarray'>
print(scores.dtype)    # int64 – all elements are 64-bit integers

# From scratch – utility constructors
zeros = np.zeros(5)     # [0. 0. 0. 0. 0.]
ones = np.ones(5)      # [1. 1. 1. 1. 1.]
rng = np.arange(0, 10, 2) # [0 2 4 6 8]
grid = np.linspace(0, 1, 6) # [0. 0.2 0.4 0.6 0.8 1.0]
```

dtype tells you exactly what's in memory. No commas in output = NumPy, not a list.

Stop Looping. Start Describing.

```
import numpy as np
scores = np.array([85, 92, 78, 95, 88])

# Python loop – don't do this
doubled_slow = []
for s in scores:
    doubled_slow.append(s * 2)    # 5 iterations, Python overhead

# NumPy vectorized – do this instead
doubled_fast = scores * 2
print(doubled_fast)  # [170 184 156 190 176]

# All of these run in compiled C – no Python loop required
print(scores + 5)      # [90 97 83 100 93]
print(scores - scores.mean()) # deviations from mean
print(scores ** 2)     # [7225 8464 6084 9025 7744]
print(np.sqrt(scores)) # [9.22 9.59 8.83 9.75 9.38]
```

One line. Entire array. Compiled execution.

Filter Without a Loop

```
import numpy as np
scores = np.array([85, 92, 78, 95, 88])

above_90 = scores > 90
print(above_90)          # [False  True False  True False]

high_scores = scores[above_90]
print(high_scores)       # [92 95]

# Same thing - one line
print(scores[scores > 90]) # [92 95]
```

`df[df["column"] > value]` – this is Pandas filtering. You already know how it works.

Stats in One Method Call → Then Outliers

```
import numpy as np
salaries = np.array([85000, 92000, 78000, 95000, 88000, 110000, 74000])

# Descriptive statistics
print(salaries.mean())          # 88857.14
print(salaries.std())           # 11580.51
print(salaries.min(), salaries.max()) # 74000 110000

# Percentiles – IQR outlier detection
q25 = np.percentile(salaries, 25) # 83000.0
q75 = np.percentile(salaries, 75) # 93500.0
iqr = q75 - q25                    # 10500.0
upper_fence = q75 + 1.5 * iqr      # 109250.0
lower_fence = q25 - 1.5 * iqr      # 67250.0

outliers = salaries[(salaries > upper_fence) | (salaries < lower_fence)]
print(outliers) # [110000] ← flagged, automatically
```


Rows, Columns, and the Axis Rule

```
import numpy as np
data = np.array([
    [85, 92, 78, 95],    # student 1
    [70, 88, 91, 84],    # student 2
    [93, 76, 89, 77]     # student 3
])

print(data.shape)      # (3, 4) - 3 rows, 4 columns
print(data[0, 2])      # 78 - row 0, column 2
print(data[:, 2])      # [78 91 89] - all rows, column 2

column_means = data.mean(axis=0)  # [82.67 85.33 86.    85.33]
row_means    = data.mean(axis=1)  # [87.5  83.25 83.75]
```

↓ axis=0 -
collapses rows

→ axis=1 -
collapses columns

axis=0 down · axis=1
across.
Repeat until automatic.

THE MENTAL SHIFT

Lists taught you to sequence.
Dicts taught you to map.
NumPy teaches you to operate
on the whole thing at once.

Next: Pandas wraps this in column names and row labels. Everything you learned here carries forward directly.